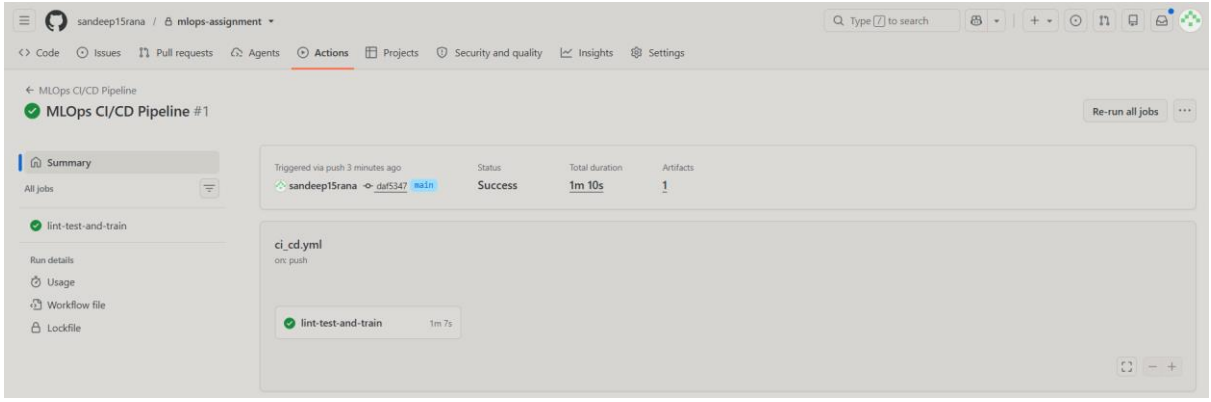
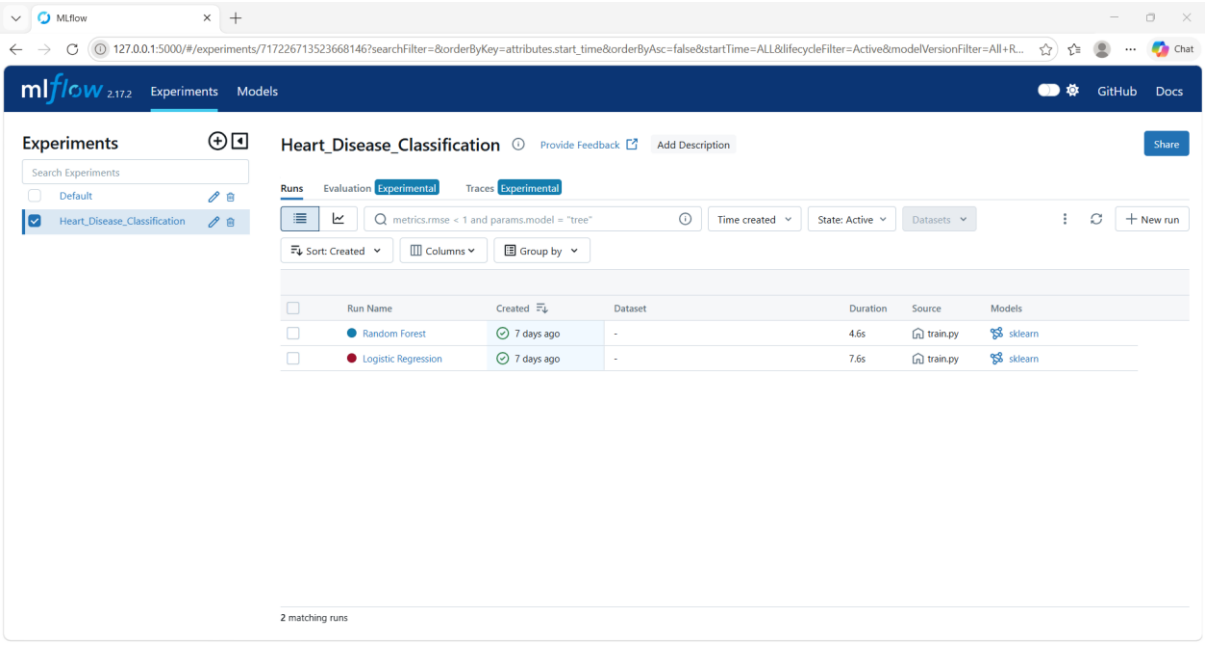
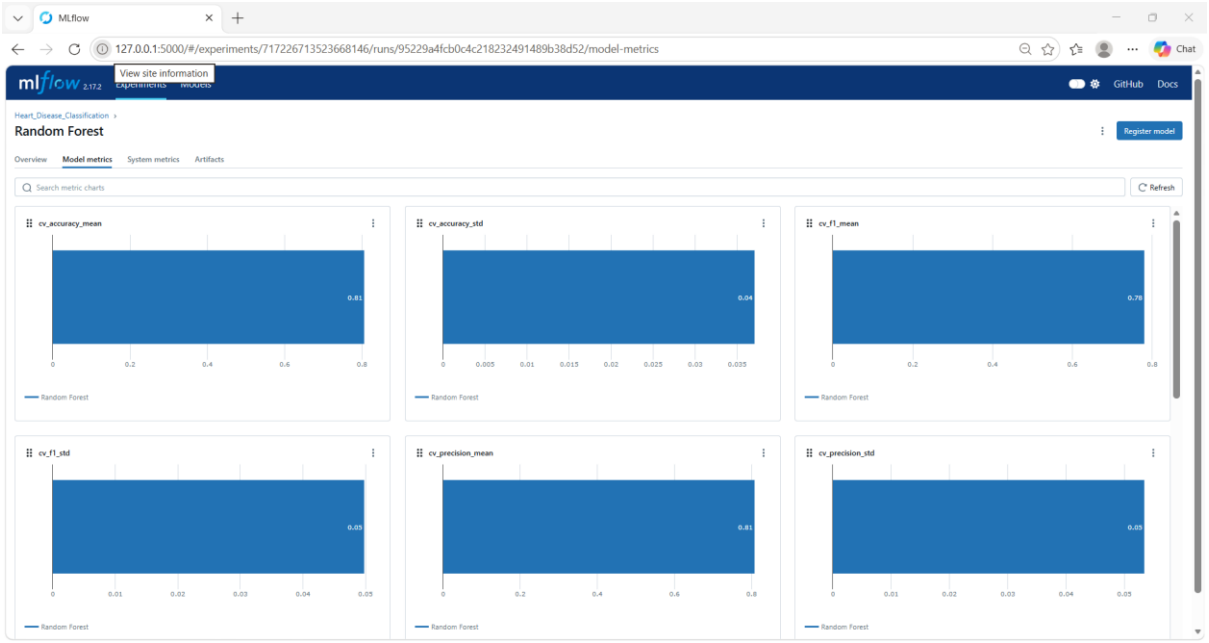
1. Push to Personal GitHub

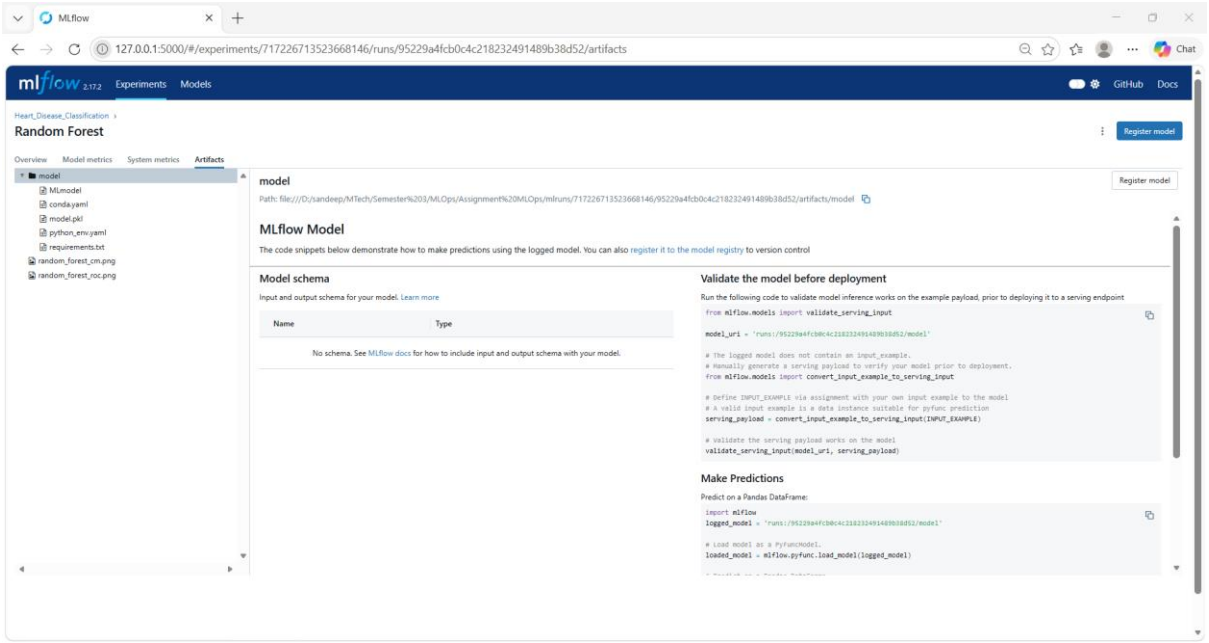


2. View MLflow Dashboard

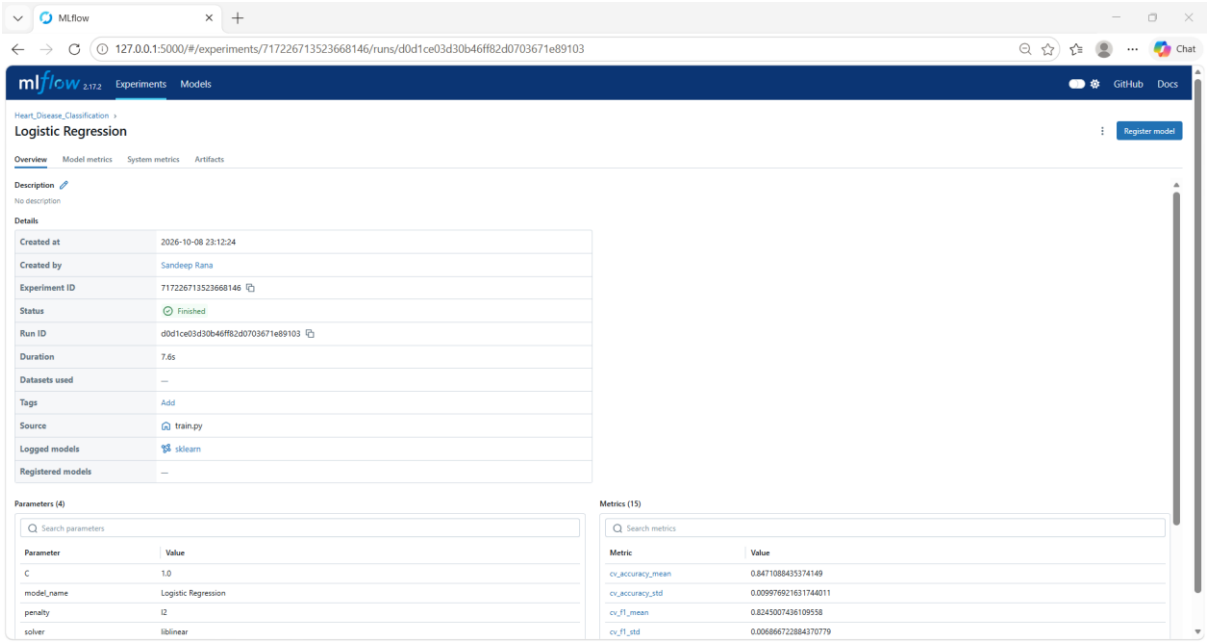


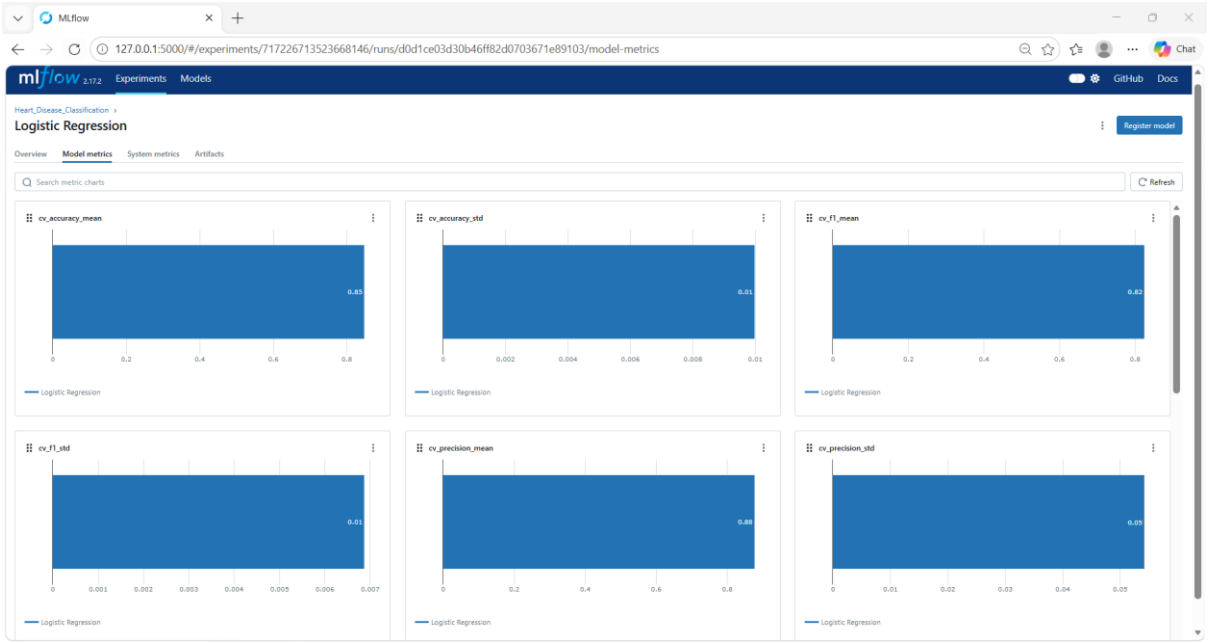
a. Random Forest

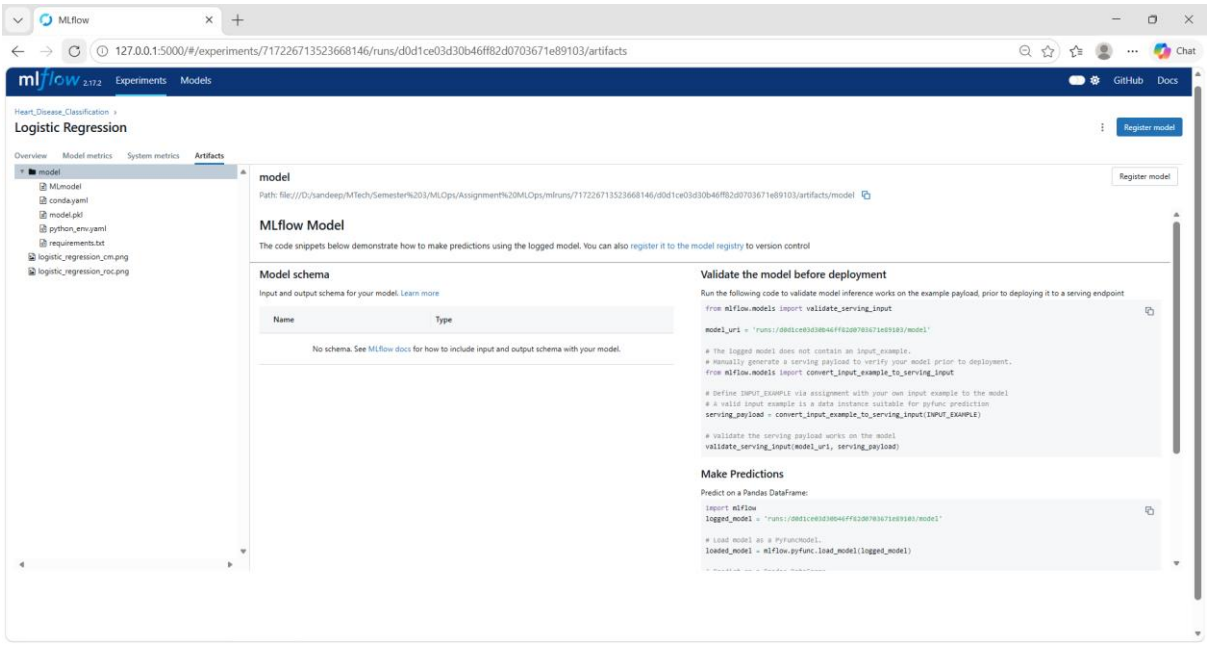




b. Logistic Regression


MLflow
127.0.0.1:5000/#/experiments/717226713523668146/runs/d0d1ce03d30b46ff82d0703671e89103
Experiments
Models
GitHub
Docs
Heart_Disease_Classification
Logistic Regression
Register model
Overview
Model metrics
System metrics
Artifacts
Description
No description
Details
Created at 2026-10-08 23:12:24
Created by Sandeep Rana
Experiment ID 717226713523668146
Status Finished
Run ID d0d1ce03d30b46ff82d0703671e89103
Duration 7.6s
Datasets used —
Tags Add
Source train.py
Logged models sklearn
Registered models —
Parameters (4)
Search parameters
Parameter Value
C 1.0
model_name Logistic Regression
penalty l2
solver liblinear
Metrics (15)
Search metrics
Metric Value
cv_accuracy_mean 0.847108843537414
cv_accuracy_std 0.009976921631744011
cv_f1_mean 0.8245007436109558
cv_f1_std 0.006866722884370779


MLflow
127.0.0.1:5000/#/experiments/717226713523668146/runs/d0d1ce03d30b46ff82d0703671e89103/model-metrics
Experiments
Models
GitHub
Docs
Heart_Disease_Classification
Logistic Regression
Register model
Overview
Model metrics
System metrics
Artifacts
Search metric charts
Refresh
cv_accuracy_mean
cv_accuracy_std
cv_f1_mean
cv_f1_std
cv_precision_mean
cv_precision_std
Logistic Regression



3. Build & Run Docker Container

```
C:\Windows\System32\cmd.e
 => => writing image sha256:32e9ed0a2247016ebe70f608513a38d525427d1f2495374cd793e8558d58f131                0.0s
 => => naming to docker.io/library/heart-disease-api:v1                                                    0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/89x3aoleoqfvn856x6o8ewvz1
PS D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>
PS D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps> docker run -d -p 8000:8000 --name heart-api heart-disease-api:v1
dbc09e21f951e2f9df6dd6207c8e96836db632e20b0d31735c96f3a08d0745d1
PS D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>
PS D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps> curl http://localhost:8000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

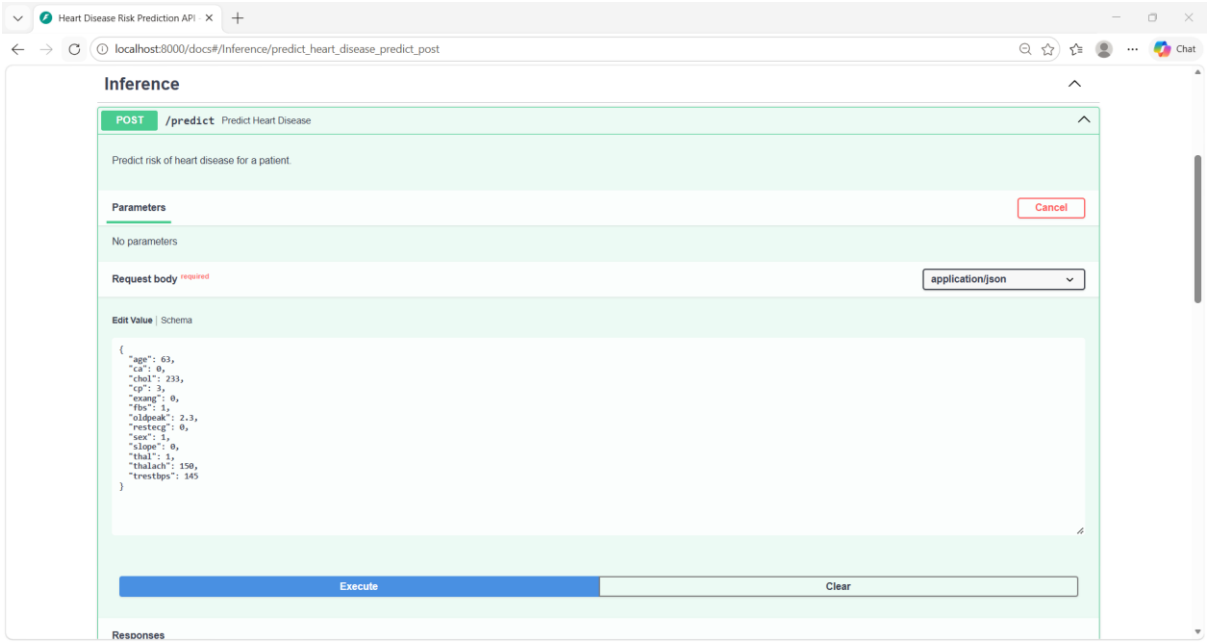
      Do you want to continue?

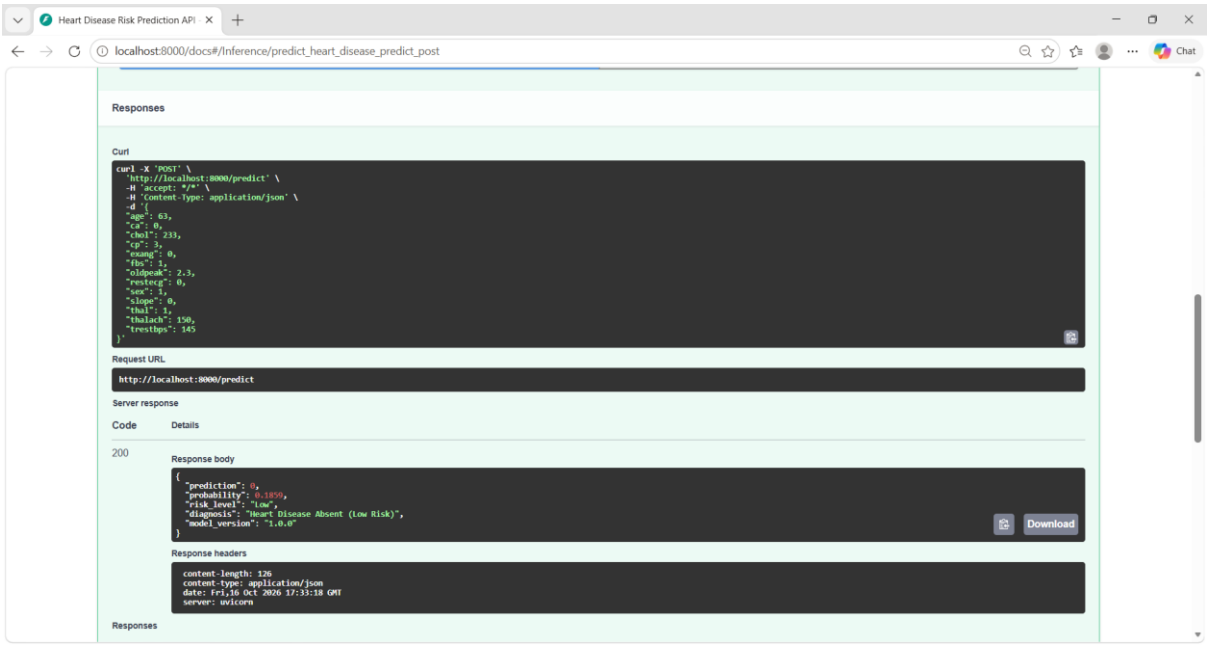
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y

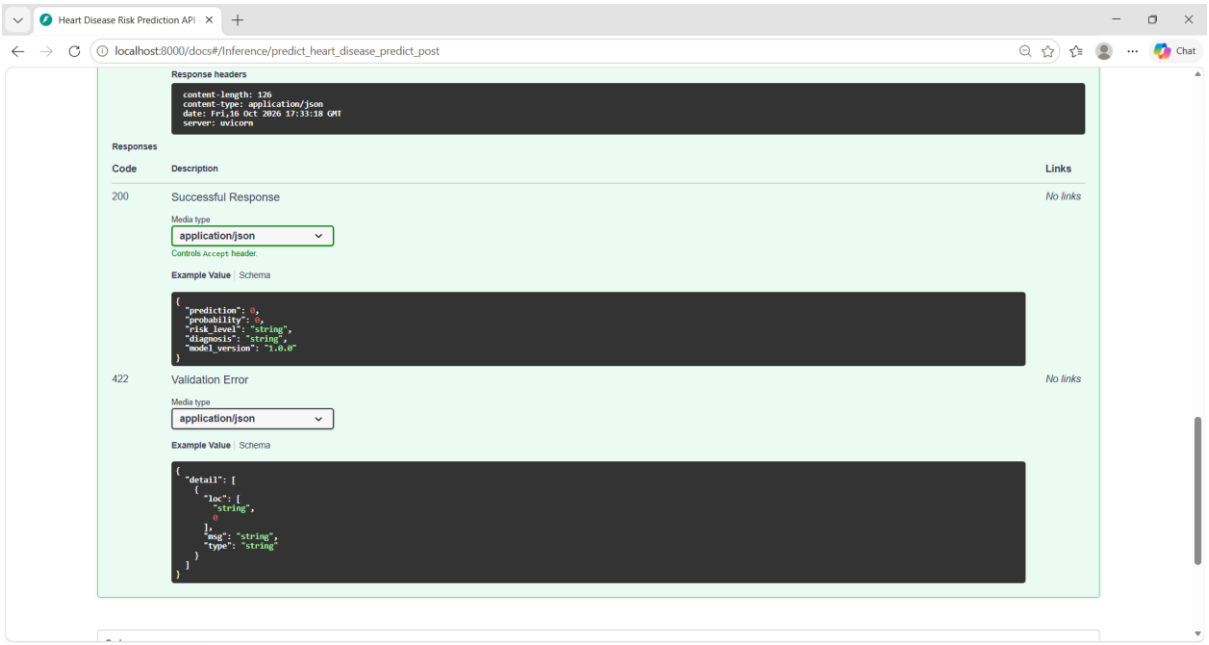

StatusCode        : 200
StatusDescription : OK
Content           : {"status":"healthy","model_loaded":true,"model_path":"models/best_model.joblib","version":"1.0.0"}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 98
                    Content-Type: application/json
                    Date: Fri, 16 Oct 2026 17:32:32 GMT
                    Server: uvicorn

                    {"status":"healthy","model_loaded":true,"model_path":"models/best_model.job...
Forms             : {}
Headers           : {[Content-Length, 98], [Content-Type, application/json], [Date, Fri, 16 Oct 2026 17:32:32 GMT], [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 98


PS D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>
```


Heart Disease Risk Prediction API
localhost:8000/docs#/Inference/predict_heart_disease_predict_post
Chat
Inference
POST
/predict
Predict Heart Disease
Predict risk of heart disease for a patient.
Parameters
Cancel
No parameters
Request body
required
application/json
Edit Value | Schema
{
"age": 63,
"ca": 0,
"chol": 233,
"cp": 3,
"exang": 0,
"fbs": 1,
"oldpeak": 2.3,
"restecg": 0,
"sex": 1,
"slope": 0,
"thal": 1,
"thalach": 150,
"trestbps": 145
}
Execute
Clear
Responses


Heart Disease Risk Prediction API
localhost:8000/docs#/Inference/predict_heart_disease_predict_post
Chat
Responses
Curl
curl -X 'POST' \
'http://localhost:8000/predict' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"age": 63,
"ca": 0,
"chol": 233,
"cp": 3,
"exang": 0,
"fbs": 1,
"oldpeak": 2.3,
"restecg": 0,
"sex": 1,
"slope": 0,
"thal": 1,
"thalach": 150,
"trestbps": 145
}'
Request URL
http://localhost:8000/predict
Server response
Code
Details
200
Response body
{
"prediction": 0,
"probability": 0.1859,
"risk_level": "Low",
"diagnosis": "Heart Disease Absent (Low Risk)",
"model_version": "1.0.0"
}
Download
Response headers
content-length: 126
content-type: application/json
date: Fri,16 Oct 2026 17:33:18 GMT
server: uvicorn
Responses



4. Run Kubernetes Local Deployment

```
D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>
D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>kubectl get pods -l app=heart-disease-api
NAME                                 READY   STATUS    RESTARTS      AGE
heart-disease-api-6b675747bf-2s2sq   1/1     Running   1 (5m6s ago)   10m
heart-disease-api-6b675747bf-g6m26   1/1     Running   1 (5m6s ago)   9m40s

D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>
D:\sandeep\MTech\Semester 3\MLOps\Assignment MLOps>kubectl get svc
NAME                    TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
heart-disease-service   LoadBalancer   10.97.63.189   <pending>     80:30830/TCP   19m
kubernetes              ClusterIP      10.96.0.1      <none>        443/TCP        19m
```

Video Recording

Part 1: Introduction & Problem Statement (0:00 – 0:30)

- **Visual on Screen:** Title slide or VS Code showing README.md.
- **Spoken Audio:**

  "Hello everyone and respected evaluators. My name is [Your Name], and this is my video presentation for MLOps Assignment 01 in course AIMLCZG523.

  The objective of this assignment is to design, develop, and deploy an end-to-end, production-ready machine learning solution following modern MLOps best practices.

  We built a binary classifier to predict the risk of heart disease using the UCI Cleveland dataset. The entire lifecycle is automated—covering data ingestion, reproducible preprocessing, experiment tracking with MLflow, automated CI/CD testing, containerization, Kubernetes deployment, and Prometheus monitoring."

Part 2: Codebase Structure & CI/CD Pipeline (0:30 – 1:15)

- **Visual on Screen:** Show the folder structure in VS Code, then switch to the GitHub repository **Actions** tab showing the green checkmark.
- **Spoken Audio:**

  "Starting with our codebase architecture, we organized the project into modular, production-ready packages:

  - src/data handles automated downloading, cleaning, and a leak-free scikit-learn ColumnTransformer preprocessing pipeline.
  - src/models contains our training pipeline, hyperparameter evaluation, and standalone inference modules.
  - src/api serves the model via FastAPI with Pydantic validation.
  - k8s/ contains our Kubernetes deployment and LoadBalancer service manifests.

  Looking at our GitHub Actions CI/CD workflow (ci_cd.yml), every push automatically triggers three automated stages:

  1. Code linting and style enforcement using ↑lake8,
  2. Automated unit testing with pytest and coverage tracking, and
  3. A verified model training step that produces and uploads pipeline artifacts. As you can see on screen, our latest pipeline run has completed with a green build."

Part 3: Automated Unit Testing Suite (1:15 – 1:45)

- **Visual on Screen:** Switch to the terminal in VS Code and execute: `powershell .venv\Scripts\python -m pytest tests/ -v`
- **Spoken Audio:**

  "Next, let us run our automated unit test suite using Pytest.

  [Press Enter and let the 9 tests run]

  All 9 tests pass cleanly. Our unit test suite validates:

  - Data downloading and missing value handling,
  - Stratified train-test split integrity,
  - Preprocessing pipeline transformation consistency,
  - Model inference bounds and valid probability outputs in range 0 to 1,
  - And FastAPI endpoints—including /health, successful /predict, and HTTP 422 error handling for invalid patient inputs."

Part 4: Experiment Tracking with MLflow (1:45 – 2:30)

- **Visual on Screen:** Switch to Browser Tab 2 showing **MLflow UI** (http://127.0.0.1:5000).
- **Spoken Audio:**

  "For experiment tracking, we integrated MLflow under the experiment Heart_Disease_Classification.

  We benchmarked two distinct classification algorithms using 5-Fold Stratified Cross-Validation on the training set:

  1. A Logistic Regression classifier with L2 regularization, and
  2. A Random Forest ensemble classifier.

  As seen in the comparison table:

  - Logistic Regression achieved a 5-Fold CV ROC-AUC of 0.9025 and a Test ROC-AUC of 0.9665, with an outstanding clinical recall of 92.86%.
  - Random Forest achieved a 0.8951 CV ROC-AUC and 0.9426 Test ROC-AUC.

  [Click on the Logistic Regression run and scroll down to Artifacts]

  For every experiment run, MLflow automatically logged all hyperparameters, cross-validation metrics, the complete serialized scikit-learn pipeline, and diagnostic plots—including the Confusion Matrix and ROC Curve. Because Logistic Regression provided superior clinical recall and ROC-AUC, it was selected and exported to models/best_model.joblib for production serving."

Part 5: Production Deployment & Live Inference (2:30 – 3:20)

- **Visual on Screen:** Switch to terminal and run: `powershell kubectl get pods -l app=heart-disease-api kubectl get svc heart-disease-service` Then switch to Browser Tab 3 showing **FastAPI Swagger UI** (/docs).
- **Spoken Audio:**

  "For production deployment, the application is packaged in a lightweight Docker container and deployed to Kubernetes.

  In the terminal, running kubectl get pods shows our deployment actively running two healthy replicas with CPU and memory limits, as well as configured liveness and readiness health probes.

  The service is exposed via a LoadBalancer.

  [Switch to Swagger UI at /docs]

  Let's test live inference using the interactive Swagger UI. Under POST /predict, we submit a sample patient payload containing age, blood pressure, cholesterol, resting ECG, and clinical test features.

  [Click 'Try it out' -> 'Execute']

  The API processes the request through our bundled preprocessing pipeline and returns a response in under 30 milliseconds:

  - Prediction: 0 (Heart Disease Absent),
  - Risk Probability: 0.1859,
  - And Clinical Risk Category: Low."

Part 6: Logging & Monitoring (3:20 – 3:45)

- **Visual on Screen:** Switch to Browser Tab 4 showing /metrics, then briefly show logs/api.log in VS Code.
- **Spoken Audio:**

  "For observability and production monitoring:

  - We integrated prometheus-fastapi-instrumentator. Calling /metrics exposes real-time Prometheus metrics, including HTTP request counts, response latency histograms, and status codes.
  - In addition, every request, duration, client IP, and prediction outcome is structured and written to logs/api.log for auditing and compliance."

Part 7: Conclusion (3:45 – 4:00)

- **Visual on Screen:** Return to the GitHub repository homepage or closing slide.
- **Spoken Audio:**

  "To conclude, this project demonstrates a complete, automated, and reproducible MLOps lifecycle—from data ingestion and MLflow experiment tracking to containerized Kubernetes deployment, automated CI/CD testing, and live monitoring.

  All source code, manifests, tests, and documentation are available in the repository. Thank you very much!"